

UNIVERSITY OF LONDON

INTERNATIONAL PROGRAMMES

BSc Computer Science and Related Subjects



CM3020 ARTIFICIAL INTELLIGENCE

MIDTERM COURSEWORK

PART B

Word Count: 2072

Author : Devon Maya Xavier
UOL Student Number : 200618744
SIM Student Number : 10204322

TABLE OF CONTENTS

1. Basic Experiments: Genetic Algorithm Implementation	3
1.1 Environment Integration & Setup	3
1.2 Fitness Function Design	4
1.3 Basic Parameter Experiments	5
2. Basic Experiments Results	6
2.1 Observations.....	6
Outliers – “High Achievers”	6
Invalid Creature Simulation	6
2.2 Analysis	7
Fitness (Population vs. Convergence Speed)	7
Top Parameter Combinations—“BASIC Potential Climbers”	8
3. Advanced Experiments	9
3.1 Motor Controls Experiment	9
3.2 Body Shape Experiment.....	9
4. Advanced Experiment Results	10
4.1 Comparative Performance	10
4.2 Creature Changes	10
5. Conclusion.....	11
APPENDICES.....	12
APPENDIX A: Sample of Table Documenting Run Results	12
APPENDIX B: Fitness Function Code.....	12
CODE.....	14
climb_ga.py **ALL OWN CODE**	14
creature.py **OWN CODE IN GREEN**	21
environment.py **IMPORTED MOUNTAIN & ARENA FROM CW-ENVT.PY**	25
genome.py **OWN CODE IN GREEN**	28
population.py **OWN CODE IN GREEN**	38
realtime_from_csv.py **OWN CODE IN GREEN**	39
simulation.py **OWN CODE IN GREEN**	42
END	46

1. BASIC EXPERIMENTS: GENETIC ALGORITHM IMPLEMENTATION

1.1 ENVIRONMENT INTEGRATION & SETUP

The first thing I did to improve modularity and make the codebase easier to understand was to transfer the environment setup (i.e., mountain and arena) from *cw-envt.py* into a new file called *environment.py*, while keeping the rest of the fitness function and evolution code in a separate file called *climb_ga.py*. This separation allows environment-related logic to be handled independently, making the overall structure cleaner and more maintainable.

Also, the creatures were initially spawning from the sky at coordinates (0, 0, 10), which caused them to bounce upon impact and distorted the fitness evaluation results. To create a more realistic and fair climbing test, I adjusted the spawn position to the side of the mountain at (6, 0, 0.5). This placement ensures that creatures begin at ground level and must genuinely climb, rather than gain height through falling.

In my analysis run, the creature moved from a distance of 6.0 to 5.635 units from the mountain center—indicating actual approach—while maintaining a low, realistic height of 0.11. The fitness score of 2.404 reflected a reasonable baseline. After implementing this change, I also set up a simple race scenario (**Figure 1**) among three creatures to test their initial performance and verify the fairness of different spawn positions.

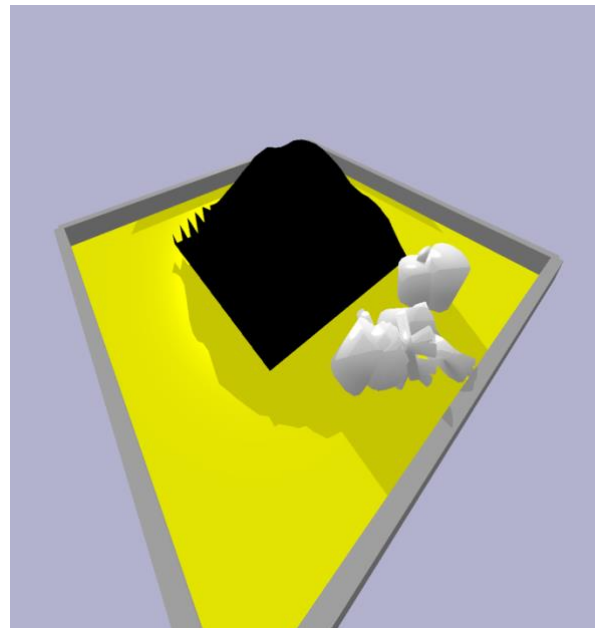


Figure 1: "Race" with 3 creatures

1.2 FITNESS FUNCTION DESIGN

In this project, I modified the fitness evaluation strategy to better reflect the goal of encouraging mountain climbing behaviour rather than simply measuring how far a creature travels. Instead of relying on the original generic *get_distance_travelled()* function in *creature.py*, which might reward random movement or bouncing, I designed a fitness function that integrates several meaningful metrics tailored to the climbing task.

The key components of the fitness function include¹

- **Height Gained:**
 - Measures how much vertical progress the creature made during the simulation. This reflects actual climbing success and is weighted strongly.
- **Horizontal Progress Toward Mountain Center:**
 - Encourages creatures to move closer to the mountain peak, which is assumed to be near the center (0, 0) of the arena. It penalizes movement away from the mountain.
- **Stability:**
 - Based on the creature's final orientation, creatures that maintain a more upright and stable pose (i.e., low tilt) are rewarded. Unstable or tumbling behaviour is penalized.
- **Efficiency:**
 - How effective the creature combines progress with time, promoting smarter rather than purely chaotic movement.
- **Terrain Bonuses:**
 - **On-Mountain Bonus:** Rewards creatures that are close to the mountain and stable.
 - **True Climbing Bonus:** Heavily rewards vertical gain *while* staying close to the mountain and remaining stable.
 - **Mountain Contact Bonus:** Rewards any contact with the mountain to promote engagement.
 - **Sustained Contact Bonus:** Further encourages creatures that remain in contact with the mountain over time.
- **Penalties:**
 - Unstable creatures have their fitness heavily reduced.
 - Creatures that move away from the mountain receive a penalty on horizontal progress.

This approach is designed to encourage realistic climbing behaviour. Rather than rewarding distance alone (which could be gained by bouncing or rolling), it prioritizes vertical movement, proximity to the mountain, balance, and deliberate interaction with the terrain—metrics more aligned with actual climbing performance.

¹ Refer to **Appendix B: Fitness Function Code** for code snippet.

1.3 BASIC PARAMETER EXPERIMENTS

I used the method of logging each run and its respective parameters using an Excel file² to systematically track performance trends and fine-tune the evolution settings. Over the course of these experiments, I varied key parameters (**Figure 2**), such as **population size**, **genome size**, **mutation rates**, and **simulation duration**, to understand their impact on climbing behaviour.

```
# Evolution parameters
GA_PARAMS = {
    'population_size': 100,
    'gene_count': 6,
    'generations': 300,
    'point_mutation_rate': 0.12,
    'point_mutation_amount': 0.05,
    'grow_mutation_rate': 0.15,
    'shrink_mutation_rate': 0.15,
    'simulation_steps': 2400,
    'elitism': True
}
```

Figure 2: Evolution Parameters Example

For population size, I tested smaller groups of **10, 20, and 30** creatures and found that larger populations, particularly at **30 and above**, led to more robust evolutionary outcomes. This eventually informed my decision to set the **final population size at 100** to maximize genetic diversity and reduce the risk of premature convergence.

I also experimented with **different genome sizes**, ranging from **3 to 10 genes**, to see how structural complexity affected behaviour. While higher gene counts sometimes resulted in higher fitness, they also produced bulky or erratic creatures. In contrast, a **gene count of 6** consistently produced realistic and stable morphologies, with **stability percentages ranging from 72% to 76%**, along with solid fitness and height scores.

For mutation rates, I tested point mutation amounts of **0.05 and 0.1** and rates ranging from **0.1 to 0.15** for both point and structural mutations. I found that a **point mutation rate of 0.12 and amount of 0.05**, combined with **grow/shrink mutation rates of 0.15**, struck a good balance between exploration and refinement.

Additionally, I tested **simulation step durations equivalent to 10 and 15 seconds** (i.e., 2400 vs. 3600 steps). The shorter 10-second simulation (2400 steps) proved to be sufficient to observe meaningful movement while keeping computation time reasonable.

Overall, these final parameters were not chosen arbitrarily but emerged through an evidence-based, iterative process that supported the potential evolution of creatures with, hopefully, increasingly competent and realistic climbing behaviours.

² Refer to **Appendix B: Sample of Table Documenting Run Results**

2. BASIC EXPERIMENTS RESULTS

2.1 OBSERVATIONS

OUTLIERS – “HIGH ACHIEVERS”

Outliers that bounced but did not climb							
Rank	Genome Size	Growth Rate	Shrink Rate	Max Best Fitness	Avg Fitness	Convergence Speed (Gen)	Stability (%)
1	10	0.18	0.15	66.08	9.89	4	67.90%
2	8	0.12	0.18	64.66	7.95	22	78.50%
3	9	0.1	0.2	35.58	7.83	23	82.10%

During the evolution runs, I occasionally noticed some unusually high fitness or height values that didn't reflect true climbing behaviour. On closer inspection, these spikes were often caused by creatures bouncing high into the air right after spawning, rather than actually scaling the mountain. This misleading behaviour occurred more frequently in creatures with higher gene counts—typically in the range of 8 to 10—likely due to their larger and more complex structures interacting unpredictably with the environment. While these outliers could potentially skew the data, they were relatively rare and didn't have a major impact on the overall progression. I suppose this is a fairly typical scenario in genetic algorithm experiments, where unexpected behaviours can emerge, especially in early generations.

INVALID CREATURE SIMULATION

Twice I encountered the error message “GetBasePositionAndOrientation failed,” which upon investigation revealed that the creature I was trying to simulate was invalid. This means that my GA system had likely created a creature with a physical structure that PyBullet could not load, causing `p.loadURDF` to fail and return an invalid ID. This happened when I tried to increase my `gene_count` along with the `grow_mutation_rate`. Therefore, I concluded that having too many genes, especially when combined with a high `grow_mutation_rate`, can lead to unstable or malformed creatures that break the simulation. As a result, I decided to cap the gene count and moderate the `grow_mutation_rate` in my final parameter settings to ensure structural feasibility and simulation stability.

2.2 ANALYSIS

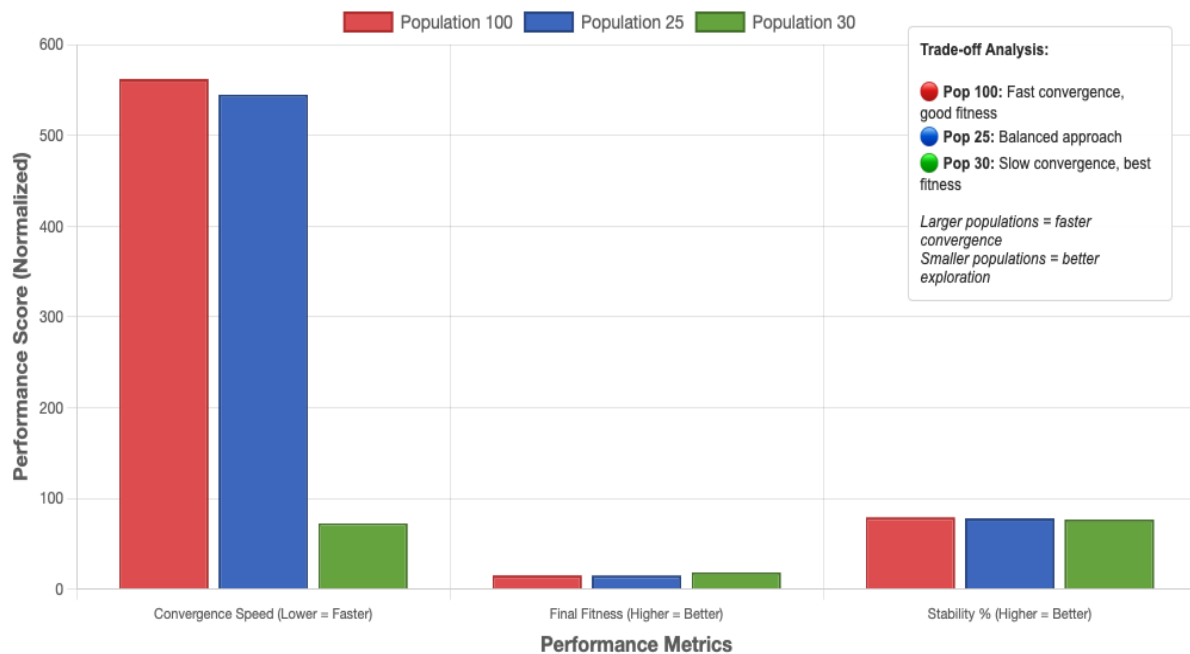
Fitness (Population Size vs. Convergence Speed)								
	Populations	Gene Count	Growth Rate	Shrink Rate	Best Total Fitness	Avg Total Fitness	Convergence Speed (Gen)	Stability
1	100	6	0.15	0.15	15.199	4.427	38	79.11%
2	25	6	0.15	0.15	15.079	4.477	55	78.06%
3	30	6	0.15	0.15	18.693	4.891	527	76.94%

FITNESS (POPULATION VS. CONVERGENCE SPEED)

This table compares how different **population sizes** influence genetic algorithm performance, holding gene count and mutation rates constant. A population size of **100** achieved strong results—converging in just **38 generations** with a max fitness of **15.199** and the highest stability (**79.11%**). This suggests large populations encourage faster convergence due to greater genetic diversity.

A smaller population of **25** also performed well, reaching **15.079** fitness in **55 generations**, with nearly identical average fitness and stability. This shows it can be a good trade-off between speed and computational efficiency.

Meanwhile, the population of **30** achieved the **highest fitness (18.693)** but took **527 generations** to converge. Although its average fitness and stability remained competitive, the slower convergence highlights the potential downside of smaller populations: more exploration but delayed adaptation.



In summary, larger populations promote faster and more stable evolution, while smaller ones may need more time to optimize but can still yield high-performing solutions.

TOP PARAMETER COMBINATIONS — "BASIC POTENTIAL CLIMBERS"

Top Parameters Combinations of "Actual Potential Climbers"								
	Populations	Gene Count	Growth Rate	Shrink Rate	Best Total Fitness	Avg Total Fitness	Generations	Stability (%)
1	30	9	0.1	0.2	15.389	6.253	120	72.44%
2	30	8	0.12	0.18	64.658	6.292	200	67.10%
3	30	8	0.1	0.15	17.265	5.843	200	51.40%
4	30	8	0.1	0.15	18.932	6.074	200	67.68%
5	25	6	0.15	0.15	16.416	5.165	300	70.57%

All five parameter settings produced creatures that showed some signs of climbing potential. They observed crawling toward the mountain, wriggling at its base, or bumping into it repeatedly. Some appeared partially stuck while trying to free themselves, and a few bounced out of the arena—likely due to excessive motor force. These actions suggest the evolutionary setup was encouraging mountain-focused behaviour.

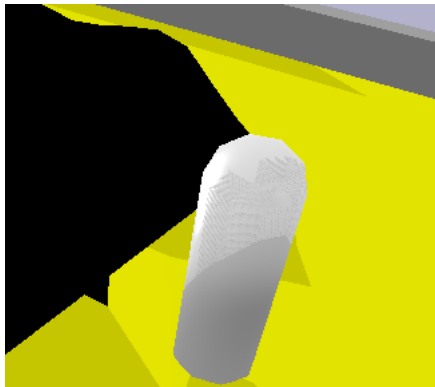


Figure 3: Cylindrical Creature

Among the settings, population sizes of 25–30 with gene counts of 8 or 9 and moderate mutation rates (0.1–0.2) gave promising results. Notably, Run 2 (population 30, gene count 8) achieved the highest fitness score of 64.658, while others like Runs 4 and 5 showed stable and consistent climbing attempts. Although larger populations help a lot with diversity, smaller ones (like in Run 5) can still produce competitive results.

In terms of evolution patterns, many creatures took on simple cylindrical forms (**Figure 3**)—this was expected, as I deliberately did not modify the genome settings for this basic experiment. The goal was to observe pure evolutionary behaviour without biasing the results through structural tweaks. While the creatures moved realistically, they often failed to ascend. Higher gene counts (8–10) sometimes led to unstable builds or unrealistic bouncing. Still, these runs showed that the system is evolving in the right direction, with parameter tuning gradually encouraging more mountain-oriented behaviors.

3. ADVANCED EXPERIMENTS

3.1 MOTOR CONTROLS EXPERIMENT

The creatures produced in the basic experiments were generally rigid and moved with slow, limited wriggling. To give them more dynamic and purposeful control, I first increased the motor amplitude by adjusting the

control_amp scale in the genome setup. I then introduced a new SAWTOOTH waveform into the Motor class, aiming to produce a “reach and pull” style motion that might help with climbing. To ensure these stronger movements didn’t compromise balance, I added *control_smoothing* for smoother transitions and better stability. Lastly, I incorporated *control_phase_offset* into the genome and Motor class, allowing different limbs to move slightly out of sync—mimicking the coordinated motion needed for climbing.

```
self.motor_type = MotorType.SAWTOOTH
self.amp = control_amp
self.freq = control_freq
self.smoothing = control_smoothing
self.phase = control_phase_offset
self.prev_output = 0
```

3.2 BODY SHAPE EXPERIMENT

In the original genome setup, creatures were limited to a single cylindrical shape, which restricted their ability to develop structures with “grip” or meaningful contact points. To address this, I expanded the shape options in the genome by introducing three new geometries: box, sphere, and capsule. This provided a balance between the stability of the box and the fluidity of the sphere, offering more opportunities for adaptive climbing behaviors to evolve. However, since box shapes can sometimes introduce unwanted rigidity—counteracting earlier efforts to improve flexibility through motor control—I modified the *to_link_element* method in the genome to generate tapered box shapes instead. This adjustment allows for the emergence of more claw-like or piton-like forms, which may help the creatures dig into the mountain surface for a more secure grip.

```
+ str(self.link_radius * self.taper) + " " + str(self.link_radius * self.taper))
```

4. ADVANCED EXPERIMENT RESULTS

4.1 COMPARATIVE PERFORMANCE

From the figures below, we can see a clear trade-off between performance and stability. The highest-performing runs (in terms of best and average fitness) come from larger populations but with slightly lower stability scores—likely due to more experimental and structurally diverse creatures evolving faster but also becoming more chaotic.

Top Parameter Combinations of "Advanced Potential Climbers"								
	Populations	Gene Count	Growth Rate	Shrink Rate	Best Total Fitness	Avg Total Fitness	Generations	Stability (%)
1	30	6	0.15	0.15	18.693	4.891	700	76.94%
2	30	6	0.15	0.15	16.9	5.058	700	75.84%
3	30	6	0.15	0.15	15.764	4.339	700	75.81%
4	100	6	0.15	0.15	21.907	8.148	300	65.32%
5	100	6	0.15	0.15	19.424	5.092	300	73.81%

Of course, parameters were tested out in a similar fashion as the basic experiments, but it is surprising to see that the introduction of enhanced motor controls and diversified body shapes did not compromise stability. In fact, the creatures demonstrated slightly improved fitness scores and maintained or even exceeded previous stability percentages despite the increased complexity in movement and structure. This suggests that the evolutionary system is robust enough to handle additional degrees of freedom without devolving into erratic or unstable behaviour—an encouraging sign that the modifications are pushing the creatures toward more functional and realistic climbing capabilities.

4.2 CREATURE CHANGES

One noticeable structural difference in the “advanced” creatures was the emergence of bicycle spoke-like limbs, which they appeared to use to manoeuvre around the environment. This was a new development in my observations and likely a direct result of introducing new body shapes into the genome. While some evolved forms still plateaued into “jumping capsule” behaviors—occasionally misleadingly inflating height scores—the presence of more appendages offered the potential for gripping or anchoring movements, especially if the terrain allowed it. In terms of movement, these creatures still often bumped repeatedly into the mountain base or crawled along it, much like their basic counterparts. However, there seemed to be a clearer sense of direction or “intent” to climb. That may be my bias, but the improvement in morphology and mechanical potential feels tangible. Overall, I believe these changes mark a clear step forward from the basic evolution phase.



Figure 4: Bicycle spoke-like limbs

5. CONCLUSION

Even though I did not get the chance to implement more advanced changes to the environment—such as adding staircases or developing more complex sensor-based control systems for the creatures, which are currently beyond my skill level—I hope to learn these techniques in the future so I can one day see the creatures successfully reach the mountain peak. This project has given me valuable exposure to the fascinating and often unexpected possibilities that genetic algorithms can offer, and it has sparked a strong interest to explore their potential even further.

APPENDICES

APPENDIX A: SAMPLE OF TABLE DOCUMENTING RUN RESULTS

Run ID	Population Size	Genome Size (gene count)	Rate	Grow Rate	Shrink Rate	Generations	Sim Time	Avg Total Fitness	Best Total Fitness	Best Height Gained	% Stable Creatures	Comments (about best height gained)
PARAMETER TUNINGS												
1	20	6	0.1	0.1	0.25	30	2400	10.976 12.343 (Generation 9)	1.724 (Generation 2)		67%	
2	20	8	0.1	0.2	0.15	50	3600	4.864 34.540 (Generation 45)	2.975 (Generation 45)		52%	
3	20	10	0.1	0.3	0.15	60	3600	5.704 15.211 (Generation 4)	1.833 (Generation 33)		65.50%	
4	20	9	0.1	0.25	0.15	60	3600	8.077 23.907 (Generation 47)	1.715 (Generation 44)		57.58%	
5	20	9	0.1	0.2	0.15	70	3600	7.322 21.574 (Generation 3)	2.306 (Generation 4)		66.29%	
6	20	9	0.1	0.2	0.15	70	2400	8.767 14.575 (Generation 7)	1.985 (Generation 37)		74.93%	
7	20	9	0.1	0.22	0.15	80	2400	8.418 13.368 (Generation 8)	1.892 (Generation 4)		65.19%	
8	20	10	0.1	0.18	0.15	90	2400	9.886 66.084 (Generation 4)	31.771 (Generation 4)		67.94%	Not accurate, too big, bounced out of arena
												Was touching mountain base then suddenly
9	30	8	0.1	0.15	0.15	90	2400	8.794 21.762 (Generation 68)	8.549 (Generation 84)		79.22%	flew out of arena
10	30	9	0.1	0.18	0.15	100	2400	6.853 22.859 (Generation 60)	2.532 (Generation 9)		67.00%	
												The best height creature was just at the
11	30	9	0.1	0.12	0.15	120	2400	6.773 22.192 (Generation 6)	2.983 (Generation 99)		74.08%	base, I don't really see it moving up..?
												The best height creature was just at the
												base, I don't really see it moving up..? It
												looked like it's stuck at the base of the
12	30	9	0.1	0.15	0.15	120	2400	8.478 20.260 (Generation 10)	4.355 (Generation 37)		79.39%	mountain or something.
												Looks promising, looks like it keeps moving
13	30	9	0.1	0.12	0.18	120	2400	9.245 13.536 (Generation 24)	2.105 (Generation 2)		84.72%	towards the mountain, good.

APPENDIX B: FITNESS FUNCTION CODE

```
#####
## Fitness Function ##
#####

def calculate_fitness(creature, steps, motors):
    start_pos = creature.start_position
    if not start_pos or not creature.last_position:
        return {
            "fitness": 0.0,
            "height_gained": 0.0,
            "is_stable": False
        }

    end_pos = creature.last_position
    end_orientation = creature.last_orientation

    # Progress-based(Core climbing metrics)
    height_gained = end_pos[2] - start_pos[2]

    # Horizontal progress toward mountain center (0,0)
    start_distance_to_mountain = ((start_pos[0])**2 + (start_pos[1])**2)**0.5
    end_distance_to_mountain = ((end_pos[0])**2 + (end_pos[1])**2)**0.5
    horizontal_progress = max(0, start_distance_to_mountain - end_distance_to_mountain)

    total_progress = (height_gained * 2.0) + (horizontal_progress * 2.0)

    # Stability(Critical for real climbing)
    euler_angles = p.getEulerFromQuaternion(end_orientation)
    roll, pitch, yaw = euler_angles
    tilt_magnitude = (roll**2 + pitch**2)**0.5

    is_stable = tilt_magnitude < 0.5
```

```

# Stability scoring
if is_stable:
    stability_score = 1.0
elif tilt_magnitude < 1.0:
    stability_score = 0.5
else:
    stability_score = 0.0

# Efficiency(Time and energy considerations)
simulation_time = steps / 240.0
efficiency = total_progress / simulation_time if simulation_time > 0 else 0

# Terrain(Mountain-specific bonuses)
on_mountain_bonus = 0
if end_distance_to_mountain < 2.0 and stability_score > 0.5:
    on_mountain_bonus = 3.0

true_climbing_bonus = 0
if (end_distance_to_mountain < 3.0 and
    height_gained > 0.3 and
    stability_score > 0.5):
    true_climbing_bonus = height_gained * 15

# Collision with mountain
mountain_contact_bonus = 0
contact_time_bonus = 0

if creature.mountain_contacts > 0:
    mountain_contact_bonus = 5.0
    num_contacts = creature.mountain_contacts
    contact_time_bonus = min(num_contacts * 0.5, 3.0)

sustained_contact_bonus = 0
if creature.mountain_contacts > 10: # creature has been in contact for at least 10 steps
    sustained_contact_bonus = creature.mountain_contacts * 0.1

```

```

# Weighted Fitness
fitness = (
    total_progress * 5.0 +
    stability_score * 4.0 +
    efficiency * 2.0 +
    on_mountain_bonus +
    true_climbing_bonus +
    mountain_contact_bonus +
    contact_time_bonus +
    sustained_contact_bonus
)

# Cost of transport
total_motor_output = 0
for m in motors:
    total_motor_output += abs(m.get_output())

cost_of_transport = total_motor_output * 0.01
fitness -= cost_of_transport

if end_pos[2] > 1.2 and end_distance_to_mountain < 2.0:
    fitness += 5.0 # true climbing bonus

# Penalties
if stability_score == 0.0:
    fitness *= 0.2

if horizontal_progress < 0:
    fitness *= 0.7

return {
    "fitness": fitness,
    "height_gained": height_gained,
    "is_stable": is_stable
}

```

CODE

climb_ga.py ****ALL OWN CODE****

```
import pybullet as p
import pybullet_data
import time
import numpy as np
import random
import creature
import population
import simulation
import genome
import math
import environment

#####
## Fitness Function ##
#####

def calculate_fitness(creature, steps, motors):
    start_pos = creature.start_position
    if not start_pos or not creature.last_position:
        return {
            "fitness": 0.0,
            "height_gained": 0.0,
            "is_stable": False
        }

    end_pos = creature.last_position
    end_orientation = creature.last_orientation

    # Progress-based(Core climbing metrics)
```

```

height_gained = end_pos[2] - start_pos[2]

# Horizontal progress toward mountain center (0,0)
start_distance_to_mountain = ((start_pos[0])**2 + (start_pos[1])**2)**0.5
end_distance_to_mountain = ((end_pos[0])**2 + (end_pos[1])**2)**0.5
horizontal_progress = max(0, start_distance_to_mountain -
end_distance_to_mountain)

total_progress = (height_gained * 2.0) + (horizontal_progress * 2.0)

# Stability (Critical for real climbing)
euler_angles = p.getEulerFromQuaternion(end_orientation)
roll, pitch, yaw = euler_angles
tilt_magnitude = (roll**2 + pitch**2)**0.5

is_stable = tilt_magnitude < 0.5

# Stability scoring
if is_stable:
    stability_score = 1.0
elif tilt_magnitude < 1.0:
    stability_score = 0.5
else:
    stability_score = 0.0

# Efficiency (Time and energy considerations)
simulation_time = steps / 240.0
efficiency = total_progress / simulation_time if simulation_time > 0 else 0

# Terrain (Mountain-specific bonuses)
on_mountain_bonus = 0
if end_distance_to_mountain < 2.0 and stability_score > 0.5:

```

```

    on_mountain_bonus = 3.0

true_climbing_bonus = 0
if (end_distance_to_mountain < 3.0 and
    height_gained > 0.3 and
    stability_score > 0.5):
    true_climbing_bonus = height_gained * 15

# Collision with mountain
mountain_contact_bonus = 0
contact_time_bonus = 0

if creature.mountain_contacts > 0:
    mountain_contact_bonus = 5.0
    num_contacts = creature.mountain_contacts
    contact_time_bonus = min(num_contacts * 0.5, 3.0)

sustained_contact_bonus = 0

if creature.mountain_contacts > 10: # creature has been in contact for at
least 10 steps
    sustained_contact_bonus = creature.mountain_contacts * 0.1

# Weighted Fitness
fitness = (
    total_progress * 5.0 +
    stability_score * 4.0 +
    efficiency * 2.0 +
    on_mountain_bonus +
    true_climbing_bonus +
    mountain_contact_bonus +
    contact_time_bonus +
    sustained_contact_bonus
)

```



```

# Cost of transport

total_motor_output = 0

for m in motors:
    total_motor_output += abs(m.get_output())

cost_of_transport = total_motor_output * 0.01
fitness -= cost_of_transport

if end_pos[2] > 1.2 and end_distance_to_mountain < 2.0:
    fitness += 5.0 # true climbing bonus

# Penalties

if stability_score == 0.0:
    fitness *= 0.2

if horizontal_progress < 0:
    fitness *= 0.7

return {
    "fitness": fitness,
    "height_gained": height_gained,
    "is_stable": is_stable
}

```

```

#####

## Evolution ##

#####

```

Evolution parameters

```
GA_PARAMS = {  
    'population_size': 100,  
    'gene_count': 6,  
    'generations': 300,  
    'point_mutation_rate': 0.12,  
    'point_mutation_amount': 0.05,  
    'grow_mutation_rate': 0.15,  
    'shrink_mutation_rate': 0.15,  
    'simulation_steps': 2400,  
    'elitism': True  
}
```

```
def main():
```

```
    print("STARTING GENETIC ALGORITHM EVOLUTION")
```

```
    print(f"Evolution Parameters:")
```

```
    for key, value in GA_PARAMS.items():
```

```
        print(f"    {key}: {value}")
```

Create evolution population

```
evo_pop = population.Population(pop_size=GA_PARAMS['population_size'],  
                                gene_count=GA_PARAMS['gene_count'])
```

Create fast headless simulation for evolution

```
evo_sim = simulation.ThreadedSim(pool_size=4)
```

Track evolution progress

```
all_time_best_fitness = -1
```

```
all_time_fittest_generation = -1
```

```

all_time_best_height = -1
all_time_highestest_generation = -1
total_stable_creatures = 0

print(f"\nStarting evolution for {GA_PARAMS['generations']} generations...")

for generation in range(GA_PARAMS['generations']):
    print(f"\n--- Generation {generation + 1} ---")

    # Evaluate all creatures using fast simulation
    print("Evaluating creatures...")
    evo_sim.eval_population(evo_pop, GA_PARAMS['simulation_steps'])

    # Calculate fitness scores and other stats
    fitness_details = [calculate_fitness(cr, GA_PARAMS['simulation_steps'],
cr.get_motors()) for cr in evo_pop.creatures]

    fitness_scores = [d['fitness'] for d in fitness_details]
    heights = [d['height_gained'] for d in fitness_details]
    stable_count = sum(1 for d in fitness_details if d['is_stable'])
    total_stable_creatures += stable_count

    # Track statistics for this generation
    avg_fitness = np.mean(fitness_scores)
    best_fitness_this_gen = np.max(fitness_scores)
    best_height_this_gen = np.max(heights)

    if best_fitness_this_gen > all_time_best_fitness:
        all_time_best_fitness = best_fitness_this_gen
        all_time_fittest_generation = generation + 1

    if best_height_this_gen > all_time_best_height:
        all_time_best_height = best_height_this_gen

```

```

    all_time_highest_generation = generation + 1

    print(f"    Avg Fitness: {avg_fitness:.3f}")
    print(f"    Best Fitness: {best_fitness_this_gen:.3f}")
    print(f"    Best Height Gained: {best_height_this_gen:.3f}")
    print(f"    Stable Creatures:
{stable_count}/{GA_PARAMS['population_size']}")

# Save elite creature

best_index = np.argmax(fitness_scores)
best_creature = evo_pop.creatures[best_index]

# Save to CSV for analysis

genome.Genome.to_csv(best_creature.dna, f"elite_gen_{generation + 1}.csv")

# Create next generation (if not last generation)

if generation < GA_PARAMS['generations'] - 1:
    print("    Creating next generation...")

    fitmap = population.Population.get_fitness_map(fitness_scores)
    new_creatures = []

    if GA_PARAMS['elitism']:
        new_creatures.append(best_creature)

    while len(new_creatures) < GA_PARAMS['population_size']:
        p1_ind = population.Population.select_parent(fitmap)
        p2_ind = population.Population.select_parent(fitmap)

        p1 = evo_pop.creatures[p1_ind]
        p2 = evo_pop.creatures[p2_ind]

        new_dna = genome.Genome.crossover(p1.dna, p2.dna)

```

```

        new_dna = genome.Genome.point_mutate(new_dna,
GA_PARAMS['point_mutation_rate'], GA_PARAMS['point_mutation_amount'])

        new_dna = genome.Genome.grow_mutate(new_dna,
GA_PARAMS['grow_mutation_rate'])

        new_dna = genome.Genome.shrink_mutate(new_dna,
GA_PARAMS['shrink_mutation_rate'])

        new_cr = creature.Creature(gene_count=len(new_dna))

        new_cr.update_dna(new_dna)

        new_creatures.append(new_cr)

    evo_pop.creatures = new_creatures

    print("EVOLUTIONARY RUN SUMMARY")
    print(f"Population Size: {GA_PARAMS['population_size']}")
    print(f"Genome Size (Gene Count): {GA_PARAMS['gene_count']}")
    print(f"Point Mutation Rate: {GA_PARAMS['point_mutation_rate']}")
    print(f"Grow/Shrink Rate:
{GA_PARAMS['grow_mutation_rate']}/{GA_PARAMS['shrink_mutation_rate']}")
    print(f"Generations: {GA_PARAMS['generations']}")
    print(f"Simulation Time per Creature:
{GA_PARAMS['simulation_steps']/240.0:.2f}s")
    print(f"Average Total Fitness of the Run: {np.mean([d['fitness'] for d in
fitness_details]):.3f}")
    print(f"Best Total Fitness of the Run: {all_time_best_fitness:.3f} (Generation
{all_time_fittest_generation})")
    print(f"Best Height Gained: {all_time_best_height:.3f} (Generation
{all_time_highest_generation})")
    print(f"Percentage of Stable Creatures: {(total_stable_creatures /
(GA_PARAMS['population_size'] * GA_PARAMS['generations'])) * 100:.2f}%")
if __name__ == '__main__':
    main()

```

creature.py ****OWN CODE IN GREEN****

```

import genome

from xml.dom.minidom import getDOMImplementation

```

```

from enum import Enum

import numpy as np

class MotorType(Enum):

    PULSE = 1

    SINE = 2

    SAWTOOTH = 3

class Motor:

    def __init__(self, control_waveform, control_amp, control_freq,
control_smoothing, control_phase_offset):

        if control_waveform <= 0.33:

            self.motor_type = MotorType.PULSE

        elif control_waveform <= 0.66:

            self.motor_type = MotorType.SINE

        else:

            self.motor_type = MotorType.SAWTOOTH

        self.amp = control_amp

        self.freq = control_freq

        self.smoothing = control_smoothing

        self.phase = control_phase_offset

        self.prev_output = 0

    def get_output(self):

        self.phase = (self.phase + self.freq) % (np.pi * 2)

        raw_output = 0

        if self.motor_type == MotorType.PULSE:

            if self.phase < np.pi:

                raw_output = 1

            else:

                raw_output = -1

```

```

if self.motor_type == MotorType.SINE:
    raw_output = np.sin(self.phase)

if self.motor_type == MotorType.SAWTOOTH:
    raw_output = (self.phase / (2 * np.pi)) * 2 - 1

    smoothed_output = (1 - self.smoothing) * raw_output + self.smoothing *
self.prev_output
    self.prev_output = smoothed_output

return smoothed_output

```

```

class Creature:

```

```

    def __init__(self, gene_count):
        self.spec = genome.Genome.get_gene_spec()
        self.dna = genome.Genome.get_random_genome(len(self.spec), gene_count)
        self.flat_links = None
        self.exp_links = None
        self.motors = None
        self.start_position = [0,0,0]
        self.last_position = [0,0,0]
        self.last_orientation = [0,0,0,1]
        self.mountain_contacts = 0

```

```

    def get_flat_links(self):
        if self.flat_links == None:
            gdicts = genome.Genome.get_genome_dicts(self.dna, self.spec)
            self.flat_links = genome.Genome.genome_to_links(gdicts)
        return self.flat_links

```

```

    def get_expanded_links(self):
        self.get_flat_links()

```

```

    if self.exp_links is not None:
        return self.exp_links

    exp_links = [self.flat_links[0]]
    genome.Genome.expandLinks(self.flat_links[0],
                               self.flat_links[0].name,
                               self.flat_links,
                               exp_links)

    self.exp_links = exp_links
    return self.exp_links

def to_xml(self):
    self.get_expanded_links()
    domimpl = getDOMImplementation()
    adom = domimpl.createDocument(None, "start", None)
    robot_tag = adom.createElement("robot")
    for link in self.exp_links:
        robot_tag.appendChild(link.to_link_element(adom))
    first = True
    for link in self.exp_links:
        if first:
            first = False
            continue
        robot_tag.appendChild(link.to_joint_element(adom))
    robot_tag.setAttribute("name", "devbot")
    return '<?xml version="1.0"?>' + robot_tag.toprettyxml()

def get_motors(self):
    self.get_expanded_links()
    if self.motors == None:
        motors = []
        for i in range(1, len(self.exp_links)):

```



```

        l = self.exp_links[i]

        m = Motor(l.control_waveform, l.control_amp, l.control_freq,
l.control_smoothing, l.control_phase_offset)

        motors.append(m)

    self.motors = motors

    return self.motors

```

```

def update_position(self, pos):
    if self.start_position == None:
        self.start_position = pos
    else:
        self.last_position = pos

```

```

def get_distance_travelled(self):
    if self.start_position is None or self.last_position is None:
        return 0

    p1 = np.asarray(self.start_position)
    p2 = np.asarray(self.last_position)
    dist = np.linalg.norm(p1-p2)
    return dist

```

```

def update_dna(self, dna):
    self.dna = dna

    self.flat_links = None

    self.exp_links = None

    self.motors = None

    self.start_position = [0,0,0]
    self.last_position = [0,0,0]
    self.last_orientation = [0,0,0,1]
    self.mountain_contacts = 0

```

```

environment.py **IMPORTED MOUNTAIN & ARENA FROM CW-ENVT.PY**

```

```

import pybullet as p

```

```

import math

import random

def make_mountain(num_rocks=100, max_size=0.25, arena_size=10, mountain_height=5,
physicsClientId=0):

    def gaussian(x, y, sigma=arena_size/4):

        """Return the height of the mountain at position (x, y) using a Gaussian
function."""

        return mountain_height * math.exp(-((x**2 + y**2) / (2 * sigma**2)))

    for _ in range(num_rocks):

        x = random.uniform(-1 * arena_size/2, arena_size/2)

        y = random.uniform(-1 * arena_size/2, arena_size/2)

        z = gaussian(x, y) # Height determined by the Gaussian function

        # Adjust the size of the rocks based on height. Higher rocks (closer to
the peak) will be smaller.

        size_factor = 1 - (z / mountain_height)

        size = random.uniform(0.1, max_size) * size_factor

        orientation = p.getQuaternionFromEuler([random.uniform(0, 3.14),
random.uniform(0, 3.14), random.uniform(0, 3.14)])

        rock_shape = p.createCollisionShape(p.GEOM_BOX, halfExtents=[size, size,
size], physicsClientId=physicsClientId)

        rock_visual = p.createVisualShape(p.GEOM_BOX, halfExtents=[size, size,
size], rgbColor=[0.5, 0.5, 0.5, 1], physicsClientId=physicsClientId)

        rock_body = p.createMultiBody(baseMass=0,
baseCollisionShapeIndex=rock_shape, baseVisualShapeIndex=rock_visual,
basePosition=[x, y, z], baseOrientation=orientation,
physicsClientId=physicsClientId)

def make_rocks(num_rocks=100, max_size=0.25, arena_size=10, physicsClientId=0):

    for _ in range(num_rocks):

        x = random.uniform(-1 * arena_size/2, arena_size/2)

        y = random.uniform(-1 * arena_size/2, arena_size/2)

        z = 0.5 # Adjust based on your needs

        size = random.uniform(0.1,max_size)

        orientation = p.getQuaternionFromEuler([random.uniform(0, 3.14),
random.uniform(0, 3.14), random.uniform(0, 3.14)])

```

```

        rock_shape = p.createCollisionShape(p.GEOM_BOX, halfExtents=[size, size,
size], physicsClientId=physicsClientId)

        rock_visual = p.createVisualShape(p.GEOM_BOX, halfExtents=[size, size,
size], rgbColor=[0.5, 0.5, 0.5, 1], physicsClientId=physicsClientId)

        rock_body = p.createMultiBody(baseMass=0,
baseCollisionShapeIndex=rock_shape, baseVisualShapeIndex=rock_visual,
basePosition=[x, y, z], baseOrientation=orientation,
physicsClientId=physicsClientId)

def make_arena(arena_size=10, wall_height=1, physicsClientId=0):

    wall_thickness = 0.5

    floor_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX,
halfExtents=[arena_size/2, arena_size/2, wall_thickness],
physicsClientId=physicsClientId)

    floor_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX,
halfExtents=[arena_size/2, arena_size/2, wall_thickness], rgbColor=[1, 1, 0, 1],
physicsClientId=physicsClientId)

    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=floor_collision_shape,
baseVisualShapeIndex=floor_visual_shape, basePosition=[0, 0, -wall_thickness],
physicsClientId=physicsClientId)

    wall_collision_shape_ew = p.createCollisionShape(shapeType=p.GEOM_BOX,
halfExtents=[arena_size/2, wall_thickness/2, wall_height/2],
physicsClientId=physicsClientId)

    wall_visual_shape_ew = p.createVisualShape(shapeType=p.GEOM_BOX,
halfExtents=[arena_size/2, wall_thickness/2, wall_height/2], rgbColor=[0.7, 0.7,
0.7, 1], physicsClientId=physicsClientId) # Gray walls

    # Create east/west walls

    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape_ew,
baseVisualShapeIndex=wall_visual_shape_ew, basePosition=[0, arena_size/2,
wall_height/2], physicsClientId=physicsClientId)

    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape_ew,
baseVisualShapeIndex=wall_visual_shape_ew, basePosition=[0, -arena_size/2,
wall_height/2], physicsClientId=physicsClientId)

    wall_collision_shape_ns = p.createCollisionShape(shapeType=p.GEOM_BOX,
halfExtents=[wall_thickness/2, arena_size/2, wall_height/2],
physicsClientId=physicsClientId)

    wall_visual_shape_ns = p.createVisualShape(shapeType=p.GEOM_BOX,
halfExtents=[wall_thickness/2, arena_size/2, wall_height/2], rgbColor=[0.7, 0.7,
0.7, 1], physicsClientId=physicsClientId) # Gray walls

```

```

# Create north/south walls

p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape_ns,
baseVisualShapeIndex=wall_visual_shape_ns, basePosition=[arena_size/2, 0,
wall_height/2], physicsClientId=physicsClientId)

p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape_ns,
baseVisualShapeIndex=wall_visual_shape_ns, basePosition=[-arena_size/2, 0,
wall_height/2], physicsClientId=physicsClientId)

```

genome.py ****OWN CODE IN GREEN****

```

import numpy as np

import copy

import random

class Genome():

    @staticmethod
    def get_random_gene(length):

        gene = np.array([np.random.random() for i in range(length)])

        return gene

    @staticmethod
    def get_random_genome(gene_length, gene_count):

        genome = [Genome.get_random_gene(gene_length) for i in range(gene_count)]

        return genome

    @staticmethod
    def get_gene_spec():

        gene_spec = {"link-shape":{"scale":1},
                    "link-length": {"scale":2},
                    "link-radius": {"scale":0.5},
                    "link-recurrence": {"scale":3},
                    "link-mass": {"scale":0.5},
                    "joint-type": {"scale":1},
                    "joint-parent":{"scale":1},
                    "joint-axis-xyz": {"scale":1},
                    "joint-origin-rpy-1":{"scale":np.pi * 4},

```

```

        "joint-origin-rpy-2":{"scale":np.pi * 4},
        "joint-origin-rpy-3":{"scale":np.pi * 4},
        "joint-origin-xyz-1":{"scale":0.5},
        "joint-origin-xyz-2":{"scale":0.5},
        "joint-origin-xyz-3":{"scale":0.5},
        "control-waveform":{"scale":1},
        "control-amp":{"scale":1.0},
        "control-freq":{"scale":1},
        "control-smoothing": {"scale": 0.1},
        "taper": {"scale": 1.0},
        "control_phase_offset": {"scale": 2*np.pi}
    }
    ind = 0
    for key in gene_spec.keys():
        gene_spec[key]["ind"] = ind
        ind = ind + 1
    return gene_spec

```

@staticmethod

```

def get_gene_dict(gene, spec):
    gdict = {}
    for key in spec:
        ind = spec[key]["ind"]
        scale = spec[key]["scale"]
        gdict[key] = gene[ind] * scale
    return gdict

```

@staticmethod

```

def get_genome_dicts(genome, spec):
    gdicts = []
    for gene in genome:

```

```

        gdicts.append(Genome.get_gene_dict(gene, spec))

    return gdicts

@staticmethod
def expandLinks(parent_link, uniq_parent_name, flat_links, exp_links):
    children = [l for l in flat_links if l.parent_name == parent_link.name]
    sibling_ind = 1
    for c in children:
        for r in range(int(c.recur)):
            sibling_ind = sibling_ind + 1
            c_copy = copy.copy(c)
            c_copy.parent_name = uniq_parent_name
            uniq_name = c_copy.name + str(len(exp_links))
            #print("exp: ", c.name, " -> ", uniq_name)
            c_copy.name = uniq_name
            c_copy.sibling_ind = sibling_ind
            exp_links.append(c_copy)

            assert c.parent_name != c.name, "Genome::expandLinks: link joined
to itself: " + c.name + " joins " + c.parent_name

            Genome.expandLinks(c, uniq_name, flat_links, exp_links)

@staticmethod
def genome_to_links(gdicts):
    links = []
    link_ind = 0
    parent_names = [str(link_ind)]
    for gdict in gdicts:
        link_name = str(link_ind)
        parent_ind = gdict["joint-parent"] * len(parent_names)
        assert parent_ind < len(parent_names), "genome.py: parent ind too
high: " + str(parent_ind) + "got: " + str(parent_names)
        parent_name = parent_names[int(parent_ind)]
        #print("available parents: ", parent_names, "chose", parent_name)

```

```

recur = gdict["link-recurrence"]
link = URDFLink(name=link_name,
                 parent_name=parent_name,
                 recur=recur+1,
                 link_length=gdict["link-length"],
                 link_radius=gdict["link-radius"],
                 link_shape=gdict["link-shape"],
                 link_mass=gdict["link-mass"],
                 joint_type=gdict["joint-type"],
                 joint_parent=gdict["joint-parent"],
                 joint_axis_xyz=gdict["joint-axis-xyz"],
                 joint_origin_rpy_1=gdict["joint-origin-rpy-1"],
                 joint_origin_rpy_2=gdict["joint-origin-rpy-2"],
                 joint_origin_rpy_3=gdict["joint-origin-rpy-3"],
                 joint_origin_xyz_1=gdict["joint-origin-xyz-1"],
                 joint_origin_xyz_2=gdict["joint-origin-xyz-2"],
                 joint_origin_xyz_3=gdict["joint-origin-xyz-3"],
                 control_waveform=gdict["control-waveform"],
                 control_amp=gdict["control-amp"],
                 control_freq=gdict["control-freq"],
                 control_smoothing=gdict["control-smoothing"],
                 taper=gdict["taper"],
                 control_phase_offset=gdict["control_phase_offset"])
links.append(link)

if link_ind != 0:# don't re-add the first link
    parent_names.append(link_name)
link_ind = link_ind + 1

# now just fix the first link so it links to nothing
links[0].parent_name = "None"
return links

```

```

@staticmethod
def crossover(g1, g2):
    x1 = random.randint(0, len(g1)-1)
    x2 = random.randint(0, len(g2)-1)
    g3 = np.concatenate((g1[x1:], g2[x2:]))
    if len(g3) > len(g1):
        g3 = g3[0:len(g1)]
    return g3

@staticmethod
def point_mutate(genome, rate, amount):
    new_genome = copy.copy(genome)
    for gene in new_genome:
        for i in range(len(gene)):
            if random.random() < rate:
                gene[i] += amount
                if gene[i] >= 1.0:
                    gene[i] = 0.9999
                if gene[i] < 0.0:
                    gene[i] = 0.0
    return new_genome

@staticmethod
def shrink_mutate(genome, rate):
    if len(genome) == 1:
        return copy.copy(genome)
    if random.random() < rate:
        ind = random.randint(0, len(genome)-1)
        new_genome = np.delete(genome, ind, 0)
        return new_genome
    else:

```



```
    return copy.copy(genome)
```

```
@staticmethod
```

```
def grow_mutate(genome, rate):
```

```
    MAX_GENOME_LENGTH = 12 #limit grow
```

```
    if len(genome) >= MAX_GENOME_LENGTH:
```

```
        return copy.copy(genome)
```

```
    if random.random() < rate:
```

```
        gene = Genome.get_random_gene(len(genome[0]))
```

```
        new_genome = copy.copy(genome)
```

```
        new_genome = np.append(new_genome, [gene], axis=0)
```

```
        return new_genome
```

```
    else:
```

```
        return copy.copy(genome)
```

```
@staticmethod
```

```
def to_csv(dna, csv_file):
```

```
    csv_str = ""
```

```
    for gene in dna:
```

```
        for val in gene:
```

```
            csv_str = csv_str + str(val) + ","
```

```
        csv_str = csv_str + '\n'
```

```
    with open(csv_file, 'w') as f:
```

```
        f.write(csv_str)
```

```
@staticmethod
```

```
def from_csv(filename):
```

```
    csv_str = ''
```

```
    with open(filename) as f:
```

```
        csv_str = f.read()
```

```
    dna = []
```

```

lines = csv_str.split('\n')
for line in lines:
    vals = line.split(',')
    gene = [float(v) for v in vals if v != '']
    if len(gene) > 0:
        dna.append(gene)
return dna

```

```

class URDFLink:
    def __init__(self, name, parent_name, recur,
        link_length=0.1,
        link_radius=0.1,
        link_shape=0.1,
        link_mass=0.1,
        joint_type=0.1,
        joint_parent=0.1,
        joint_axis_xyz=0.1,
        joint_origin_rpy_1=0.1,
        joint_origin_rpy_2=0.1,
        joint_origin_rpy_3=0.1,
        joint_origin_xyz_1=0.1,
        joint_origin_xyz_2=0.1,
        joint_origin_xyz_3=0.1,
        control_waveform=0.1,
        control_amp=0.1,
        control_freq=0.1,
        control_smoothing=0.1,
        taper=0.1,
        control_phase_offset=0.1):
        self.name = name
        self.parent_name = parent_name
        self.recur = recur

```

```

self.link_length=link_length
self.link_radius=link_radius
self.link_shape=link_shape
self.link_mass=link_mass
self.joint_type=joint_type
self.joint_parent=joint_parent
self.joint_axis_xyz=joint_axis_xyz
self.joint_origin_rpy_1=joint_origin_rpy_1
self.joint_origin_rpy_2=joint_origin_rpy_2
self.joint_origin_rpy_3=joint_origin_rpy_3
self.joint_origin_xyz_1=joint_origin_xyz_1
self.joint_origin_xyz_2=joint_origin_xyz_2
self.joint_origin_xyz_3=joint_origin_xyz_3
self.control_waveform=control_waveform
self.control_amp=control_amp
self.control_freq=control_freq
self.control_smoothing=control_smoothing
self.taper=taper
self.control_phase_offset = control_phase_offset
self.sibling_ind = 1

```

```

def to_link_element(self, adom):
    link_tag = adom.createElement("link")
    link_tag.setAttribute("name", self.name)
    vis_tag = adom.createElement("visual")
    geom_tag = adom.createElement("geometry")

    if self.link_shape < 0.33:
        shape_tag = adom.createElement("box")
        shape_tag.setAttribute("size", str(self.link_length) + " " +
str(self.link_radius * self.taper) + " " + str(self.link_radius * self.taper))
    elif self.link_shape < 0.66:
        shape_tag = adom.createElement("capsule")

```

```

        shape_tag.setAttribute("length", str(self.link_length))
        shape_tag.setAttribute("radius", str(self.link_radius))
    else:
        shape_tag = adom.createElement("sphere")
        shape_tag.setAttribute("radius", str(self.link_radius))

    geom_tag.appendChild(shape_tag)
    vis_tag.appendChild(geom_tag)

    coll_tag = adom.createElement("collision")
    c_geom_tag = adom.createElement("geometry")

    if self.link_shape < 0.33:
        c_shape_tag = adom.createElement("box")
        c_shape_tag.setAttribute("size", str(self.link_length) + " " +
str(self.link_radius * self.taper) + " " + str(self.link_radius * self.taper))
    elif self.link_shape < 0.66:
        c_shape_tag = adom.createElement("capsule")
        c_shape_tag.setAttribute("length", str(self.link_length))
        c_shape_tag.setAttribute("radius", str(self.link_radius))
    else:
        c_shape_tag = adom.createElement("sphere")
        c_shape_tag.setAttribute("radius", str(self.link_radius))

    c_geom_tag.appendChild(c_shape_tag)
    coll_tag.appendChild(c_geom_tag)

    inertial_tag = adom.createElement("inertial")
    mass_tag = adom.createElement("mass")
    # pi r^2 * height
    mass = np.pi * (self.link_radius * self.link_radius) * self.link_length
    mass_tag.setAttribute("value", str(mass))
    inertia_tag = adom.createElement("inertia")

```

```
        # <inertia ixx="0.0003" iyy="0.0003" izz="0.0003" ixy="0" ixz="0"
        iyz="0"/>
```

```
        inertia_tag.setAttribute("ixx", "0.03")
```

```
        inertia_tag.setAttribute("iyy", "0.03")
```

```
        inertia_tag.setAttribute("izz", "0.03")
```

```
        inertia_tag.setAttribute("ixy", "0")
```

```
        inertia_tag.setAttribute("ixz", "0")
```

```
        inertia_tag.setAttribute("iyz", "0")
```

```
        inertial_tag.appendChild(mass_tag)
```

```
        inertial_tag.appendChild(inertia_tag)
```

```
        link_tag.appendChild(vis_tag)
```

```
        link_tag.appendChild(coll_tag)
```

```
        link_tag.appendChild(inertial_tag)
```

```
        return link_tag
```

```
def to_joint_element(self, adom):
```

```
    joint_tag = adom.createElement("joint")
```

```
    joint_tag.setAttribute("name", self.name + "_to_" + self.parent_name)
```

```
    joint_tag.setAttribute("type", "revolute")
```

```
    parent_tag = adom.createElement("parent")
```

```
    parent_tag.setAttribute("link", self.parent_name)
```

```
    child_tag = adom.createElement("child")
```

```
    child_tag.setAttribute("link", self.name)
```

```
    axis_tag = adom.createElement("axis")
```

```
    if self.joint_axis_xyz <= 0.33:
```

```
        axis_tag.setAttribute("xyz", "1 0 0")
```

```
    if self.joint_axis_xyz > 0.33 and self.joint_axis_xyz <= 0.66:
```

```
        axis_tag.setAttribute("xyz", "0 1 0")
```

```
    if self.joint_axis_xyz > 0.66:
```

```
        axis_tag.setAttribute("xyz", "0 0 1")
```

```

    limit_tag = adom.createElement("limit")
    # effort upper lower velocity
    limit_tag.setAttribute("effort", "1")
    limit_tag.setAttribute("upper", "-3.1415")
    limit_tag.setAttribute("lower", "3.1415")
    limit_tag.setAttribute("velocity", "1")
    # <origin rpy="0 0 0" xyz="0 0.5 0"/>
    orig_tag = adom.createElement("origin")

    rpy1 = self.joint_origin_rpy_1 * self.sibling_ind
    rpy = str(rpy1) + " " + str(self.joint_origin_rpy_2) + " " +
    str(self.joint_origin_rpy_3)

    orig_tag.setAttribute("rpy", rpy)
    xyz = str(self.joint_origin_xyz_1) + " " + str(self.joint_origin_xyz_2) +
    " " + str(self.joint_origin_xyz_3)
    orig_tag.setAttribute("xyz", xyz)

    joint_tag.appendChild(parent_tag)
    joint_tag.appendChild(child_tag)
    joint_tag.appendChild(axis_tag)
    joint_tag.appendChild(limit_tag)
    joint_tag.appendChild(orig_tag)
    return joint_tag

```

population.py ****OWN CODE IN GREEN****

```

import creature
import numpy as np

```

```

class Population:
    def __init__(self, pop_size, gene_count):
        self.creatures = [creature.Creature(
                                gene_count=gene_count)
                            for i in range(pop_size)]

    @staticmethod
    def get_fitness_map(fits):
        fitmap = []
        total = 0
        min_fit = np.min(fits)
        for f in fits:
            total = total + f - min_fit
            fitmap.append(total)
        return fitmap

    @staticmethod
    def select_parent(fitmap):
        r = np.random.rand() # 0-1
        r = r * fitmap[-1]
        for i in range(len(fitmap)):
            if r <= fitmap[i]:
                return i

```

realtime_from_csv.py ****OWN CODE IN GREEN****

```

import os
import genome
import sys

```

```

import creature
import pybullet as p
import time
import random
import numpy as np

## ... usual starter code to create a sim and floor
import environment

def main(csv_file):
    assert os.path.exists(csv_file), "Tried to load " + csv_file + " but it does not exist"

    p.connect(p.GUI)
    p.setPhysicsEngineParameter(enableFileCaching=0)
    p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0)
    p.resetDebugVisualizerCamera(cameraDistance=15, cameraYaw=45, cameraPitch=-30, cameraTargetPosition=[0,0,0])
    p.setGravity(0, 0, -10)

    # Create the environment
    arena_size = 20
    environment.make_arena(arena_size=arena_size)
    p.setAdditionalSearchPath('shapes/')
    mountain = p.loadURDF("gaussian_pyramid.urdf", (0, 0, 0.5),
    p.getQuaternionFromEuler((0,0,0)), useFixedBase=1)

    # generate a random creature
    dna = genome.Genome.from_csv(csv_file)
    cr = creature.Creature(gene_count=len(dna))
    cr.update_dna(dna)

```



```

# save it to XML
with open('test.urdf', 'w') as f:
    f.write(cr.to_xml())
# load it into the sim
rob1 = p.loadURDF('test.urdf')
# place it at the start
p.resetBasePositionAndOrientation(rob1, [6, 0, 0.5], [0, 0, 0, 1])
start_pos, orn = p.getBasePositionAndOrientation(rob1)

# iterate
elapsed_time = 0
wait_time = 1.0/240 # seconds
total_time = 30 # seconds
step = 0
while True:
    p.stepSimulation()
    step += 1
    if step % 24 == 0:
        motors = cr.get_motors()
        assert len(motors) == p.getNumJoints(rob1), "Something went wrong"
        for jid in range(p.getNumJoints(rob1)):
            mode = p.VELOCITY_CONTROL
            vel = motors[jid].get_output()
            p.setJointMotorControl2(rob1,
                                    jid,
                                    controlMode=mode,
                                    targetVelocity=vel)
            new_pos, orn = p.getBasePositionAndOrientation(rob1)
            #print(new_pos)
            dist_moved = np.linalg.norm(np.asarray(start_pos) -
                                         np.asarray(new_pos))
            time.sleep(wait_time)
            elapsed_time += wait_time

```

```

        if elapsed_time > total_time:
            break

if __name__ == "__main__":
    assert len(sys.argv) == 2, "Usage: python playback_test.py csv_filename"
    main(sys.argv[1])

```

simulation.py ****OWN CODE IN GREEN****

```

import pybullet as p
from multiprocessing import Pool

class Simulation:
    def __init__(self, sim_id=0):
        self.sim_id = sim_id
        self.physicsClientId = -1

    def run_creature(self, cr, iterations=2400):
        try:
            self.physicsClientId = p.connect(p.DIRECT)
            pid = self.physicsClientId
            p.resetSimulation(physicsClientId=pid)
            p.setPhysicsEngineParameter(enableFileCaching=0, physicsClientId=pid)

            p.setGravity(0, 0, -10, physicsClientId=pid)

            import environment
            arena_size = 20
            environment.make_arena(arena_size=arena_size, physicsClientId=pid)

            mountain_position = (0, 0, 0.5)
            mountain_orientation = p.getQuaternionFromEuler((0, 0, 0))
            p.setAdditionalSearchPath('shapes/', physicsClientId=pid)

```

```

        mountain = p.loadURDF("gaussian_pyramid.urdf",
                                mountain_position,
                                mountain_orientation,
                                useFixedBase=1,
                                physicsClientId=pid)

        cr.start_position = [6,0,0.5]
        xml_file = 'temp' + str(self.sim_id) + '.urdf'
        xml_str = cr.to_xml()
        with open(xml_file, 'w') as f:
            f.write(xml_str)

        cid = p.loadURDF(xml_file, physicsClientId=pid)
        if cid < 0:
            raise Exception("Failed to load URDF")

        p.resetBasePositionAndOrientation(cid, cr.start_position, [0, 0, 0,
1], physicsClientId=pid)

        for step in range(iterations):
            p.stepSimulation(physicsClientId=pid)
            if step % 24 == 0:
                self.update_motors(cid=cid, cr=cr)

            pos, orn = p.getBasePositionAndOrientation(cid,
physicsClientId=pid)
            cr.update_position(pos)

            cr.last_orientation = orn
            cr.last_position = pos

            cr.mountain_contacts = len(p.getContactPoints(bodyA=cid,
bodyB=mountain, physicsClientId=pid))

        except Exception as e:

```

```

        # This handles creatures that are not valid URDFs
        cr.last_position = cr.start_position
        cr.last_orientation = [0,0,0,1]
        cr.mountain_contacts = 0
    finally:
        if self.physicsClientId >= 0:
            p.disconnect(self.physicsClientId)

def update_motors(self, cid, cr):
    """
    cid is the id in the physics engine
    cr is a creature object
    """
    for jid in range(p.getNumJoints(cid,
                                    physicsClientId=self.physicsClientId)):
        m = cr.get_motors()[jid]

        p.setJointMotorControl2(cid, jid,
                                controlMode=p.VELOCITY_CONTROL,
                                targetVelocity=m.get_output(),
                                force = 5,
                                physicsClientId=self.physicsClientId)

# You can add this to the Simulation class:
def eval_population(self, pop, iterations):
    for cr in pop.creatures:
        self.run_creature(cr, 2400)

class ThreadedSim():
    def __init__(self, pool_size):

```

```

        self.sims = [Simulation(i) for i in range(pool_size)]

    @staticmethod
    def static_run_creature(sim, cr, iterations):
        sim.run_creature(cr, iterations)
        return cr

    def eval_population(self, pop, iterations):
        """
        pop is a Population object
        iterations is frames in pybullet to run for at 240fps
        """
        pool_args = []
        start_ind = 0
        pool_size = len(self.sims)
        while start_ind < len(pop.creatures):
            this_pool_args = []
            for i in range(start_ind, start_ind + pool_size):
                if i == len(pop.creatures):# the end
                    break
                # work out the sim ind
                sim_ind = i % len(self.sims)
                this_pool_args.append([
                    self.sims[sim_ind],
                    pop.creatures[i],
                    iterations]
                )
            pool_args.append(this_pool_args)
            start_ind = start_ind + pool_size

        new_creatures = []
        for pool_argset in pool_args:

```

```
        with Pool(pool_size) as p:
            # it works on a copy of the creatures, so receive them
            creatures = p.starmap(ThreadedSim.static_run_creature,
pool_argset)
            # and now put those creatures back into the main
            # self.creatures array
            new_creatures.extend(creatures)
        pop.creatures = new_creatures
```

****END****